

Dossier de soutenance

RNCP37827BC01

Réaliser la collecte, le stockage et la mise à disposition des données d'un projet en intelligence artificielle

PPMT

Plateforme Prédictive des Métiers en Tension — Île-de-France

Candidate	Claire Lucie DIOUF
Formation	Développeur en Intelligence Artificielle — Artefact
Projet	Pipeline de données et API sur les offres d'emploi franciliennes (France Travail + Adzuna)
Coéquipier	Bernard GBOHOUGNON
Données	Collecte du 6 octobre 2026 : 62 435 offres (France Travail + Adzuna)
Dépôt	github.com/Clairelucie/PPMT
Soutenance	16 octobre 2026

Sommaire

1. Présentation du service numérique.....	3
2. Architecture générale du pipeline.....	4
3. C1 — Collecter les données de manière automatisée.....	5
4. C2 — Préparer les données pour leur première utilisation.....	8
5. C3 — Mettre en œuvre des règles d'agrégation de données.....	12
6. C4 — Créer une base de données.....	15
7. C5 — Mettre à disposition les données via une API.....	18
8. Tests et qualité.....	22
9. Service numérique et ouverture Machine Learning.....	23
10. Bilan et perspectives.....	25
Annexes.....	27

1. Présentation du service numérique

Contexte et problématique

Selon l'enquête Besoins en Main-d'Œuvre (BMO) 2025 de France Travail, 41,7 % des projets de recrutement franciliens sont jugés difficiles par les employeurs (50,1 % au niveau national). Les informations sur les métiers qui recrutent sont pourtant dispersées : chaque site d'emploi publie ses propres offres, avec ses propres catégories, et les statistiques officielles paraissent une fois par an.

PPMT (Plateforme Prédictive des Métiers en Tension) rassemble les offres d'emploi publiées en Île-de-France par **France Travail** et par l'agrégateur **Adzuna**, les harmonise sur le référentiel des métiers **ROME**, et calcule pour chaque métier un **indice de tension** qui mesure la pression de recrutement exercée par les employeurs.

Problématique

Comment collecter, fiabiliser et mettre à disposition, de façon automatisée, les offres d'emploi franciliennes de plusieurs sources afin d'identifier les métiers qui recrutent le plus et d'orienter candidats, recruteurs et organismes de formation ?

Ce que fait le service

- **Collecte** des offres via deux API (France Travail, Adzuna), relançable à la demande et planifiable chaque jour, et deux fichiers de référence géographiques : les contours des départements (carte, densité) et la liste des communes d'Île-de-France (pour retrouver le département de chaque offre).
- **Nettoyage et harmonisation** : dédoublonnage, salaires en € bruts annuels (les salaires non affichés sont complétés par une médiane), contrats normalisés, départements, masquage des e-mails et des téléphones.
- **Stockage** dans une base relationnelle SQLite de 4 tables, avec contraintes d'intégrité et indicateurs agrégés en SQL.
- **Mise à disposition** par une **API REST FastAPI** sécurisée (7 endpoints, documentation Swagger) et par un **dashboard Streamlit** de 6 onglets.

Utilisateurs cibles

Utilisateur	Usage de PPMT
Cabinets de recrutement	Repérer les métiers où la concurrence entre employeurs est forte ; ajuster délais et honoraires
Candidats et conseillers emploi	Savoir quels métiers recrutent réellement, dans quel département, avec quel type de contrat
Organismes de formation	Aligner l'offre de formation (CPF, OPCO) sur les métiers les plus demandés
Salariés en recherche de mobilité (CEP)	Repérer les métiers qui recrutent et leur niveau de tension pour préparer une mobilité ou une reconversion, avec un conseiller en évolution professionnelle (CEP)
Développeurs / data analysts	Interroger les données via l'API pour les intégrer à leurs propres outils

Mon rôle dans le projet

Le projet est mené en binôme avec Bernard GBOHOUGNON. **Chacun a un rôle identifié.** J'ai pris en charge la collecte France Travail (authentification OAuth2, découpage en cascade), le stockage en modules testables, l'API, les tests automatisés, le dashboard Streamlit et la partie Machine Learning. J'ai également créé le dépôt Git et

j'en assure la gestion (branches, commits, déploiement du dashboard). Bernard a collecté et nettoyé les données Adzuna et mis en place l'idempotence et la journalisation du pipeline.

2. Architecture générale du pipeline

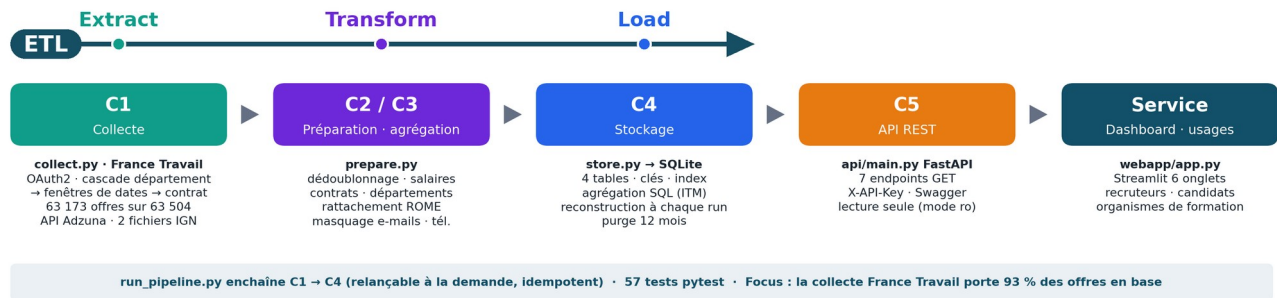


Figure 1 — Chaîne de traitement PPMT, des sources au service

Chaque compétence du bloc correspond à un module Python indépendant, testable et exécutable seul. La chaîne suit une logique ETL : extraction des offres par API (collect.py), transformation en Python avant le chargement (prepare.py), chargement dans SQLite (store.py) ; les indicateurs par métier sont ensuite agrégés en SQL dans la base. `run_pipeline.py` enchaîne les étapes C1 → C4 ; il se lance à la demande (python `run_pipeline.py --collect`) ; il est planifiable (ligne cron documentée), mais la planification quotidienne n'est pas mise en place : c'est une perspective (§10). Relancé sur les mêmes fichiers, il redonne le même résultat (idempotence, §3 et §6). L'API (C5) lit la base en lecture seule (connexion SQLite ouverte en mode « ro ») : seul le pipeline écrit dans la base, l'API n'expose que des requêtes GET, et un appel ne peut donc ni modifier ni corrompre les données, ni entrer en conflit avec une reconstruction de la base ; c'est le principe du moindre privilège.

Arborescence du dépôt

PPMT/	
├─ src/	
│ ├─ collect.py	C1 – API France Travail, API Adzuna, contours et communes IDF
│ ├─ prepare.py	C2/C3 – nettoyage, normalisation, dédoublement, RGPD
│ └─ store.py	C4 – schéma SQLite, chargement offres + référentiels, indice SQL
├─ api/main.py	C5 – API REST FastAPI
├─ tests/test_pipeline.py	57 tests pytest
├─ run_pipeline.py	script qui enchaîne C1 → C4
├─ webapp/app.py	dashboard Streamlit
├─ notebook/ml_tests.py	expérimentation ML
├─ docs/ dictionnaire.md · rgpd.md	
└─ Dockerfile	brouillon, non validé

Couche	Technologies
Collecte	Python 3.11 · requests · python-dotenv · OAuth2 client_credentials
Préparation	pandas · expressions régulières · hashlib
Stockage	SQLite 3 (clés étrangères, CHECK, fonctions fenêtre)
API	FastAPI · Pydantic · Uvicorn
Tests	pytest · FastAPI TestClient (httpx)
Visualisation	Streamlit · Plotly

3. C1 — Collecter les données de manière automatisée

Sources de données

Source	Type	Accès	Contenu	Volume
France Travail — Offres d'emploi v2	API REST JSON	OAuth2 (client_id / secret)	Offres, code ROME natif, contrat, salaire, expérience	63 173 brutes (8 dépts) → 58 301 en base
Adzuna	API REST JSON	app_id / app_key	Offres agrégées de nombreux sites, catégorie, salaire	12 500 brutes (250 pages) → 4 134 en base
GeoJSON départements (IGN / Insee)	Fichier	HTTPS public	Contours des 8 départements : centre et superficie en base, carte	8 entités
Communes d'Île-de-France (IGN / Insee)	Fichier	HTTPS public	Code Insee, nom et département : localisation des offres Adzuna	1 276 communes

Les identifiants sont lus dans un fichier `.env` exclu du dépôt (`.gitignore`) ; un modèle `.env.example` documente les variables attendues.

Authentification OAuth2 et découpage en cascade (France Travail)

L'API France Travail renvoie au maximum 150 offres par appel et limite la position de départ d'une page à 3 000 (le dernier appel accepté est `range=3000-3149`) : une requête remonte donc au plus 3 150 offres, soit 21 pages de 150. Or l'Île-de-France en compte environ 63 500 (compteur Content-Range de l'API). Une simple boucle par département s'arrêterait à environ 3 000 offres par département, soit environ 24 000 offres, 38 % du total. `collect.py` découpe donc la recherche en cascade : (1) par département ; (2) si le total dépasse 3 150, par fenêtres de dates de création (365 jours, coupées en deux tant que nécessaire, jusqu'à 1 heure) ; (3) si une fenêtre d'une heure dépasse encore le plafond, par type de contrat. Les offres sont dédoublonnées par identifiant. Résultat : 63 173 offres sur 63 504 annoncées (99,5 %), en 492 appels et environ 6 min 40 s, sans erreur HTTP 429.

Adzuna annonce 220 238 offres pour l'Île-de-France et renvoie au plus 50 résultats par page (maximum de l'API). Nous avons fixé dans `collect.py` un plafond de 250 pages (constante `ADZUNA_MAX_PAGES`), soit 12 500 offres lues : c'est un choix de notre collecte, pas une limite d'Adzuna. Adzuna constitue donc un échantillon des offres, formé des premières pages renvoyées par l'API, sans tirage aléatoire ; France Travail reste la source de référence pour le volume. Relever ce plafond est une perspective (§10).

src/collect.py – obtention du jeton

```
def ft_token(session: requests.Session, client_id: str, client_secret: str) -> str:
    r = session.post(FT_TOKEN_URL, data={
        "grant_type": "client_credentials",
        "client_id": client_id,
        "client_secret": client_secret,
        "scope": "api_offresdemploi2 o2dsoffre",
    }, timeout=30)
    r.raise_for_status()
    token = r.json().get("access_token")
    if not token:
        raise RuntimeError("Token France Travail absent de la réponse")
    return token
```

Principe de la cascade France Travail (pseudo-code simplifié de collect.py)

```
def collecter_zone(zone):
    total = compter(zone)
    if total <= 3150:
        return lire_pages(zone)
    if duree(zone) > 1 heure:
        a, b = couper_en_deux(zone)
        return collecter_zone(a) + collecter_zone(b)
    return [o for c in TYPES_CONTRAT for o in lire_pages(zone, contrat=c)] # dernier recours
offres = dedoublonner_par_id(o for d in DEPARTEMENTS for o in collecter_zone(d))
```

Robustesse de la collecte

src/collect.py – reprise sur erreur

```
def get_with_retry(session: requests.Session, url: str, *, params=None, headers=None,
                  tries: int = 3, timeout: int = 30) -> requests.Response:
    """GET avec 3 tentatives et back-off exponentiel (1 s, 2 s, 4 s).
    Respecte l'en-tête Retry-After renvoyé avec un HTTP 429."""
    for attempt in range(1, tries + 1):
        try:
            r = session.get(url, params=params, headers=headers, timeout=timeout)
            if r.status_code == 429:
                wait = int(r.headers.get("Retry-After", 2 ** attempt))
                log.warning(f"HTTP 429 – pause {wait}s (tentative {attempt}/{tries})")
                time.sleep(wait)
                continue
            r.raise_for_status()
            return r
        except requests.RequestException as e:
            if attempt == tries:
                raise
            log.warning(f"Erreur réseau ({e}) – nouvelle tentative dans {2 ** (attempt - 1)}s")
            time.sleep(2 ** (attempt - 1))
    raise RuntimeError(f"Échec après {tries} tentatives : {url}")
```

Mécanisme	Mise en œuvre
Reprise sur erreur	3 tentatives, attente exponentielle (1 s, 2 s, 4 s) ; respect de l'en-tête Retry-After d'un HTTP 429 ; une source en échec n'arrête pas les autres (code retour non nul)
Respect des quotas	Pause de 100 ms entre deux pages France Travail (limite : 10 appels/s) et de 200 ms entre deux pages Adzuna
Idempotence	Un fichier brut par source et par jour ; déjà présent → non re-téléchargé (--force pour forcer) ; offres France Travail dédoublonnées par identifiant
Traçabilité	Journal horodaté logs/collect_AAAAMMJJ_HHMMSS.log, enregistré à chaque collecte (date et heure dans le nom du fichier et sur chaque ligne) et affiché à l'écran ; empreinte MD5 de chaque fichier
Isolation des échecs	Une source en échec n'arrête pas les autres ; le script se termine avec le code retour 1 si une source échoue (0 si tout va bien), ce qui permet de détecter l'échec ; aucun outil de planification n'est en place à ce jour
Contrôle des fichiers	GeoJSON : les 8 départements doivent être présents ; communes : au moins 1 200, réparties sur les 8 départements, sinon exception
Aplatissement	ft_aplatir() / adzuna_aplatir() transforment le JSON imbriqué en lignes plates au schéma attendu par C2

Exécution réelle – journal de collecte

```
$ python src/collect.py --source geojson
15:50:45 | INFO | === PPMT – Collecte C1 ===
15:50:45 | INFO | GeoJSON : 8 départements IDF → departements_idf.geojson (MD5
ec919e5ad96f6219a8d8d0e5b6ad5da7)
15:50:45 | INFO | Terminé – 1/1 sources OK – journal : collect_20260928_155045.log
$ python src/collect.py --source geojson # 2e exécution : idempotence
15:50:46 | INFO | GeoJSON : déjà présent (MD5 ec919e5ad96f6219a8d8d0e5b6ad5da7) – ignoré
```

4. C2 — Préparer les données pour leur première utilisation

Les deux sources n'ont ni le même schéma, ni les mêmes conventions. `prepare.py` applique des **règles unitaires** (une fonction = une règle, chacune testée) puis projette les deux sources sur un **schéma commun de 19 colonnes**.

Problèmes détectés et méthodes retenues

Pour chaque problème, j'ai comparé plusieurs méthodes et retenu la combinaison qui garde le plus de données tout en restant fiable.

Problème	Méthode retenue	Justification	Résultat
Salaires non affichés : 56 % des offres (69 % Adzuna, 55 % France Travail)	Médiane des salaires affichés du groupe le plus précis (≥ 3 salaires) : métier $\times$ département $\rightarrow$ métier $\rightarrow$ domaine ROME $\rightarrow$ département $\rightarrow$ Île-de-France. Colonnes <code>salaire_impute</code> et <code>salaire_source</code>	Médiane robuste aux salaires extrêmes. Le métier explique davantage le salaire que le département. Les indicateurs n'utilisent que les salaires affichés : pas d'enrichissement circulaire	100 % des offres ont un salaire. Affichés : 27 438 ; métier $\times$ département : 24 635 ; métier : 6 638 ; domaine : 2 035 ; département : 1 679 ; région : 10
Doublons : même offre publiée plusieurs fois ou sur les deux sources	URL, puis empreinte titre + entreprise + lieu, entre les deux sources ; priorité à France Travail	Une même offre a des URL différentes et peut être publiée sur les deux sources	12 404 doublons supprimés (6 757 par URL, 5 647 par empreinte) et 27 offres écartées à la préparation ; 0 doublon en base (UNIQUE). Détail source par source dans le tableau suivant
Entreprises non renseignées (24,7 % France Travail, 2,9 % Adzuna)	« Non renseigné », exclu du comptage des entreprises	Ne rien inventer ; garder <code>nb_entreprises</code> juste	0 valeur vide ; « Non renseigné » : 14 375 offres France Travail, 120 offres Adzuna
Salaires horaires, mensuels, annuels ; bornes inversées	$\times 1\,820$ h ou $\times 12$; bornes 10 k€ – 300 k€ ; min et max permutés si inversés	Comparer des montants de même nature	Bornes inversées permutées, montants hors bornes écartés. En base : 27 438 salaires affichés (26 136 France Travail, 1 302 Adzuna), tous entre 10 k€ et 300 k€, 0 minimum supérieur au maximum
Contrats hétérogènes (« CDD - 12 Mois », « permanent », « Temps plein »)	7 catégories : CDI, CDD, Intérim, Alternance, Libéral, Autre, Non renseigné ; temps de travail Adzuna dans une colonne à part	Plus précis que 4 grandes catégories, sans confondre « non renseigné » et « autre »	Croisement des deux sources possible. En base, France Travail : CDI 37 186 · CDD 9 463 · Intérim 9 298 · Libéral 1 968 · Autre 386 ; Adzuna : CDI 1 002 · CDD 854 · Non renseigné 2 278 ; temps

Problème	Méthode retenue	Justification	Résultat
			de travail Adzuna : 983 temps plein, 139 temps partiel
Codes ROME absents chez Adzuna	1. intitulé identique à des offres France Travail ; 2. catégorie Adzuna → ROME ; 3. vide. Code fourre-tout M1607 écarté	L'intitulé est plus précis que la catégorie : Adzuna classe « Comptable » en restauration	636 codes par intitulé, 1 388 par catégorie ; 2 110 offres Adzuna sans code (51 %), exclues du calcul de tension
Lieux Adzuna sans département (« Montévrain, Lagny-sur-Marne »)	Code du libellé, nom de département, puis référentiel des 1 276 communes d'Île-de-France (IGN / Insee) et pôles connus (La Défense)	Remplace les listes de villes écrites à la main, qui se trompaient (Viry-Châtillon classé dans le 92)	32 offres Adzuna restent sans département (lieu « Île-de-France » seul) ; hors région : supprimées
Coordonnées de recruteurs dans les descriptions	Masquage [email] / [tel]	RGPD : aucune coordonnée de recruteur conservée en clair	15 offres avec e-mail masqué (17 adresses) et 15 offres avec téléphone masqué en base

Deux règles plus simples complètent ce tableau. Les dates en formats mixtes sont converties en ISO 8601 (0 date hors format en base, du 9 octobre 2025 au 6 octobre 2026) ; la vue `v_offres_analyse` calcule année, mois, âge et offre récente à la volée, sans redondance. Les intitulés sont normalisés (espaces, mention « (H/F) » retirée) pour regrouper les mêmes offres, dans les deux sources.

Résultats de la préparation : volumes avant et après

Les écarts se lisent source par source, de la collecte brute (\$3) jusqu'à la base. Chaque différence est expliquée : doublons, offres écartées, purge des offres publiées plus de 365 jours avant l'offre la plus récente de la base.

Étape	France Travail	Adzuna	Total	Écart
Offres lues (brutes)	63 173	12 500	75 673	—
Après dédoublonnage et offres écartées	58 301	4 941	63 242	- 12 431 (- 16,4 %) : 12 404 doublons et 27 offres écartées
Après purge de 12 mois (en base)	58 301	4 134	62 435	- 807 (- 1,3 %), toutes Adzuna
Écart total, de la collecte à la base	- 4 872 (- 7,7 %)	- 8 366 (- 66,9 %)	- 13 238 (- 17,5 %)	Adzuna perd le plus : doublons avec France Travail, ancienneté

Contrôle du compte : $75\,673 - 12\,404 - 27 - 807 = 62\,435$ offres en base. Les 27 offres écartées ont un titre de moins de 3 caractères ou un lieu hors Île-de-France.

Qualité des champs avant et après

Critère	Avant préparation	Après préparation	Écart
Salaire renseigné	27 438 offres (43,9 %)	62 435 offres (100 %)	+ 34 997 salaires calculés, signalés par <code>salaire_impute</code>
Doublons	12 404 repérés	0 en base	contrainte UNIQUE sur <code>hash_offre</code>
Code ROME Adzuna	0 (non fourni par l'API)	2 024 offres (49 %)	2 110 sans code (51 %),

Critère	Avant préparation	Après préparation	Écart
			exclues du calcul de tension
E-mails et téléphones dans les descriptions	15 offres avec e-mail (17 adresses) et 15 avec téléphone, en clair	0 en clair	masqués en [email] et [tel]

Chiffres mesurés sur la base du 6 octobre 2026 (62 435 offres).

Exemples de règles

src/prepare.py

```
def normaliser_salaire_annuel(val):
    """Ramène un montant en € brut annuel : < 100 → horaire (×1 820 h), < 5 000 → mensuel (×12).
    Hors bornes [10 000 ; 300 000] → valeur aberrante, remplacée par NaN."""
    if val is None or pd.isna(val):
        return None
    val = float(val)
    if val < 100:
        val *= 1820
    elif val < 5000:
        val *= 12
    return val if 10_000 <= val <= 300_000 else None
```

src/prepare.py – salaires non affichés : cascade de médianes

```
def imputer_salaires(df: pd.DataFrame, min_obs: int = 3) -> tuple[pd.DataFrame, dict]:
    """Salaire non affiché → médiane des salaires AFFICHÉS du groupe le plus précis qui en compte
    au moins `min_obs` :
    1. même métier ROME dans le même département (ex. aide-soignant dans le 93)
    2. même métier ROME en Île-de-France
    3. même domaine ROME (3 premiers caractères, ex. J15 = soins)
    4. même département
    5. toute l'Île-de-France
    Combine deux approches : médiane par département puis par catégorie, et médiane par métier.
    La médiane, pas la moyenne : quelques salaires très élevés ne la déplacent pas.
    salaire_min / salaire_max restent vides (on n'invente pas de fourchette) ; salaire_impute = 1
    et
    salaire_source indiquent la valeur calculée, et les indicateurs n'utilisent que les salaires
    affichés :
    c'est ce qui évite l'« enrichissement circulaire » (des valeurs imputées qui se confirmeraient
    elles-mêmes)."""
    df = df.copy()
    for col in ("code_rome", "code_dept"):
        if col not in df:
            df[col] = None
    affiche = df["salaire_moyen"].notna()
    df["salaire_impute"] = (~affiche).astype(int)
    df["salaire_source"] = pd.Series("affiché", index=df.index).where(affiche, None)
    df["_domaine"] = df["code_rome"].str[:3]
    obs = df[affiche]
    niveaux = [(["code_rome", "code_dept"], NIVEAUX_SALAIRE[1]), (["code_rome"],
    NIVEAUX_SALAIRE[2]),
    (["_domaine"], NIVEAUX_SALAIRE[3]), (["code_dept"], NIVEAUX_SALAIRE[4])]
    for cles, nom in niveaux:
        med = obs.groupby(cles)["salaire_moyen"].agg(["median", "count"])
        med = med.loc[med["count"] >= min_obs, "median"]
        a_remplir = df["salaire_moyen"].isna() & df[cles].notna().all(axis=1)
        if not a_remplir.any() or med.empty:
            continue
        idx = pd.MultiIndex.from_frame(df.loc[a_remplir, cles]) if len(cles) > 1 else
        df.loc[a_remplir, cles[0]]
```

```

valeurs = pd.Series(med.reindex(idx).to_numpy(), index=df.index[a_remplir])
ok = valeurs.notna()
df.loc[valeurs.index[ok], "salaire_moyen"] = valeurs[ok].round(0)
df.loc[valeurs.index[ok], "salaire_source"] = nom
mediane_globale = float(obs["salaire_moyen"].median())
reste = df["salaire_moyen"].isna()
df.loc[reste, "salaire_moyen"] = round(mediane_globale)
df.loc[reste, "salaire_source"] = NIVEAUX_SALAIRE[5]
stats = df["salaire_source"].value_counts().reindex(NIVEAUX_SALAIRE, fill_value=0).to_dict()
stats["valeur_mediane_globale"] = round(mediane_globale)
return df.drop(columns="_domaine"), stats

```

Pourquoi la médiane : sur les salaires affichés en Île-de-France, la moyenne est de 34 089 € et la médiane de 29 120 €. Quelques salaires très élevés tirent la moyenne vers le haut ; la médiane représente mieux une offre typique. Les indicateurs par métier et par département sont ensuite calculés sur les seuls salaires affichés, pour que les valeurs complétées ne se confirment pas elles-mêmes.

src/prepare.py – RGPD

```

def pseudonymiser(texte) -> str:
    """RGPD : masque les e-mails et numéros de téléphone de recruteurs présents dans le texte libre."""
    if texte is None or pd.isna(texte):
        return ""
    return RE_TEL.sub("[tel]", RE_EMAIL.sub("[email]", str(texte)))

```

src/prepare.py – dédoublonnage inter-sources

```

def dedoublonner(df: pd.DataFrame) -> tuple[pd.DataFrame, dict]:
    """1) même URL 2) même empreinte. Priorité à France Travail (codes ROME natifs)."""
    n0 = len(df)
    df = df.sort_values("source", key=lambda s: s.ne("france_travail")) # FT en premier
    df = df[df["url"].isna() | ~df.duplicated("url")]
    n1 = len(df)
    df = df.drop_duplicates("hash_offre", keep="first")
    return df.reset_index(drop=True), {"doublons_url": n0 - n1, "doublons empreinte": n1 - len(df)}

```

Résultat

Sortie : [data/processed/offres_unifiees.csv](#) — **63 242 offres** (58 301 France Travail, 4 941 Adzuna) après suppression de 12 404 doublons (6 757 par URL, 5 647 par empreinte) et 27 offres écartées à la préparation (titre de moins de 3 caractères ou lieu hors Île-de-France), puis 62 435 en base après application de la fenêtre de conservation de 12 mois (807 offres purgées, §6). Le chiffre final, utilisé partout dans ce dossier, est celui de la base : 62 435 offres.

5. C3 — Mettre en œuvre des règles d'agrégation de données

Les indicateurs par métier sont calculés **en SQL, directement dans la base**, après chaque chargement des offres ; seul le salaire médian est calculé en Python (`store.calculer_medianes`), SQLite n'ayant pas de fonction médiane. Le code ROME sert de clé commune aux deux sources : natif chez France Travail, obtenu par correspondance catégorie → ROME chez Adzuna.

Règles d'agrégation

Indicateur	Règle	Justification
nb_offres_ft / adzuna / total	COUNT par source et par code ROME	Volume de la demande des employeurs
nb_entreprises	COUNT DISTINCT, « Non renseigné » exclu	Une offre massive d'un seul employeur ≠ tension de marché
nb_departements	COUNT DISTINCT code_dept	Diffusion géographique du besoin
part_cdi	Moyenne de (contrat = CDI) × 100	Qualité des emplois proposés
salaire_median	Médiane des salaires affichés (valeurs complétées exclues), calculée en Python car SQLite n'a pas de fonction MEDIAN	Attractivité du métier, sans être faussée par les salaires extrêmes
part_salaire_affiche	Part des offres qui affichent un salaire	Fiabilité du salaire médian ; transparence des employeurs
indice_tension	nb_offres_total ÷ moyenne des métiers × 100	100 = métier moyen ; comparable d'une collecte à l'autre
statut	Seuils 50 / 100 / 150	Lecture immédiate pour un public non technique

src/store.py – agrégation SQL (fonction fenêtre AVG() OVER ())

```
SQL_INDICATEURS = """
INSERT INTO indicateurs_tension
WITH agg AS (
    SELECT code_rome,
           SUM(source = 'france_travail') AS nb_offres_ft,
           SUM(source = 'adzuna') AS nb_offres_adzuna,
           COUNT(*) AS nb_offres_total,
           COUNT(DISTINCT NULLIF(entreprise, 'Non renseigné')) AS nb_entreprises,
           COUNT(DISTINCT code_dept) AS nb_departements,
           ROUND(AVG(contrat = 'CDI') * 100, 1) AS part_cdi,
           NULL AS salaire_median, -- calculée
ensuite (médiane)
           ROUND(AVG(salaire_impute = 0) * 100, 1) AS part_salaire_affiche
    FROM offres
    WHERE code_rome IS NOT NULL
    GROUP BY code_rome
)
SELECT code_rome, nb_offres_ft, nb_offres_adzuna, nb_offres_total, nb_entreprises,
       nb_departements, part_cdi, salaire_median, part_salaire_affiche,
       ROUND(nb_offres_total * 100.0 / AVG(nb_offres_total) OVER (), 1) AS indice_tension,
       CASE
           WHEN nb_offres_total * 100.0 / AVG(nb_offres_total) OVER () > 150 THEN 'TRES EN
TENSION'
           WHEN nb_offres_total * 100.0 / AVG(nb_offres_total) OVER () > 100 THEN 'EN TENSION'
```

```

        WHEN nb_offres_total * 100.0 / AVG(nb_offres_total) OVER () > 50 THEN 'EQUILIBRE'
        ELSE 'SATURÉ'
    END AS statut,
    :date_calcul
FROM agg;

```

Ce que mesure — et ne mesure pas — l'indice de tension

L'indice mesure la **pression de la demande des employeurs** : un métier à 3 842 publie 38 fois plus d'offres qu'un métier moyen (aide ménagère, K1304 : 1 531 offres contre 40 en moyenne). Il ne s'agit pas du ratio officiel offres / demandeurs d'emploi de la DARES : PPMT ne dispose pas du nombre de candidats par métier. C'est la principale piste d'enrichissement (§10).

Lecture de l'indice : 100 correspond à un métier moyen ; un indice de 1 513 signifie environ 15 fois plus d'offres que la moyenne. Sur la base finale, la moyenne est de 39,84 offres par métier (60 325 offres rattachées ÷ 1 514 métiers) : K1304 (aide ménagère), avec 1 531 offres, obtient $1\,531 \times 100 \div 39,84 \approx 3\,842$. Les statuts découlent de seuils : saturé sous 50, équilibré de 50 à 100, en tension de 100 à 150, très en tension au-delà de 150 (1 006 / 179 / 94 / 235 métiers ; 329 en tension).

Validation externe de l'indice

Pour vérifier que l'indice n'est pas arbitraire, il a été confronté à deux références de France Travail. Sur le Top 10 des métiers difficiles à recruter (enquête BMO 2025), huit métiers sur dix sont classés « très en tension » par l'indice, alors que 15,5 % seulement des métiers le sont : l'indice repère bien le niveau de tension.

En revanche, sur l'ensemble des métiers, l'indice ne reproduit pas le classement de l'indicateur officiel. Il mesure l'intensité de la demande des employeurs (le volume d'offres), alors que l'indicateur officiel dépend surtout du rapport entre offres et demandeurs : il se lit donc comme un indicateur de demande, pas comme une mesure de la difficulté à recruter. Intégrer les demandeurs inscrits par métier est la suite logique. Le détail figure dans le Rapport Qualité des Données, section 8.

Contrôles de cohérence de la base (8 octobre 2026) : 62 435 offres sans doublon ; moyenne de l'indice égale à 100 à l'arrondi près ; 1 514 métiers, dont 329 au-dessus du seuil de tension ; 2 110 offres sans code ROME (3,4 %), exclues de l'indice.

Résultats

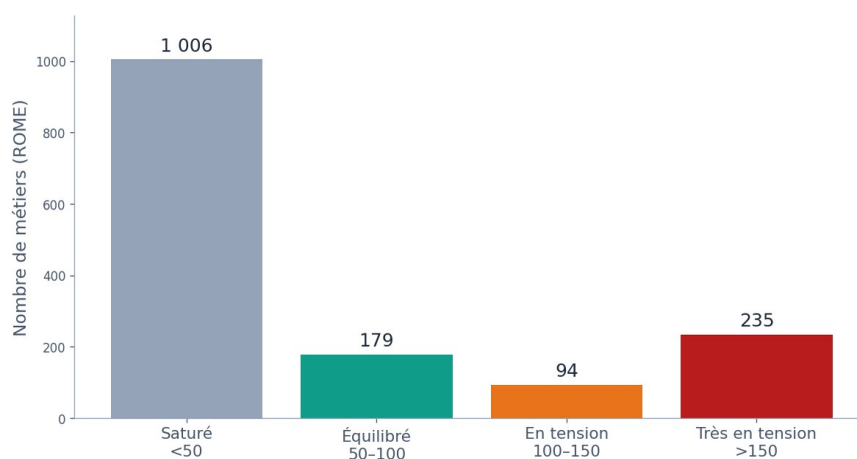


Figure 2 — Répartition des 1 514 métiers ROME par statut

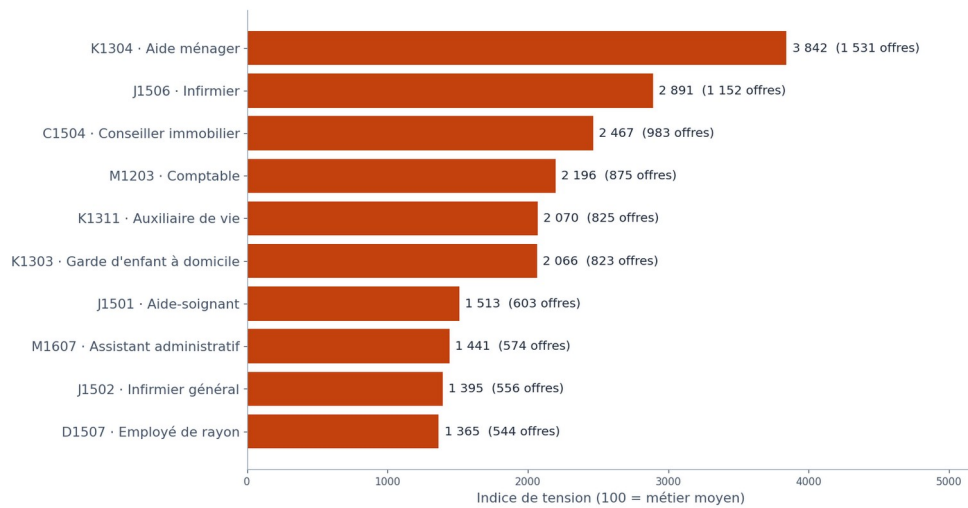


Figure 3 — Les 10 métiers à plus forte tension en Île-de-France (collecte d'octobre 2026)

Lecture : le chiffre à droite de la barre est l'indice de tension ; entre parenthèses, le nombre d'offres du métier dans la base.

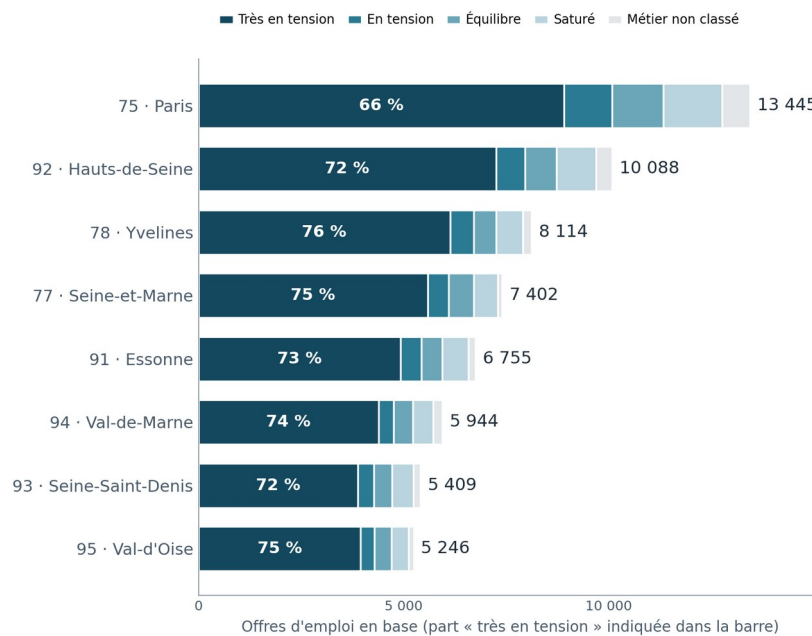


Figure 4 — Offres par département et niveau de tension des métiers (collecte d'octobre 2026)

Légende : le gris clair regroupe les offres sans code ROME, exclues de l'indice de tension.

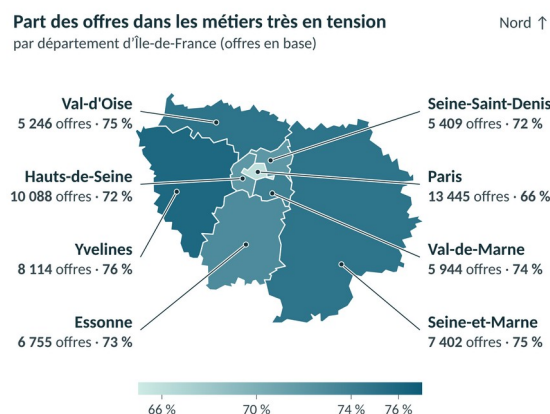


Figure 4 bis — Carte de l'Île-de-France : offres en base et part des métiers très en tension, par département (collecte d'octobre 2026)

6. C4 — Créer une base de données

Choix technologique

Besoin du projet	Réponse de SQLite
Nature des données	Tabulaires, schéma stable : modèle relationnel
Volume	≈ 62 000 offres : largement suffisant
Intégrité (clés, contraintes)	Clés étrangères, CHECK, UNIQUE
Agrégations	SQL et fonctions fenêtre (calcul de l'ITM en base)
Déploiement	Un seul fichier, aucun serveur à installer

Les offres, une fois aplaties, ont une structure fixe et sont interrogées par filtres et agrégations : un modèle relationnel est le plus adapté. SQLite offre ces garanties sans serveur à administrer, ce qui rend le projet simple à installer et à rejouer.

Modèle physique de données

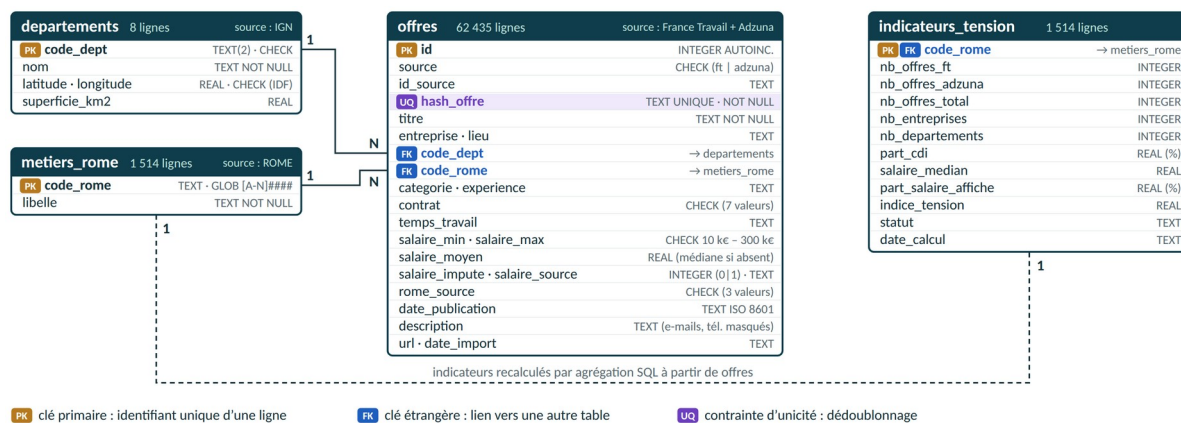


Figure 5 — Modèle physique : 2 référentiels, 1 table de faits, 1 table d'agrégats

Le modèle respecte la 3e forme normale : les libellés de départements et de métiers ne sont stockés qu'une fois, dans leurs référentiels. `indicateurs_tension` est une table dérivée, recalculée à chaque exécution à partir de `offres`. La table `departements` reçoit aussi, depuis le fichier GeoJSON, le centre géographique (latitude, longitude) et la superficie de chaque département, calculés dans `store.py` : l'API s'en sert pour la densité d'offres, le dashboard pour la carte.

`src/store.py` – table des offres

```
CREATE TABLE IF NOT EXISTS offres (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  source      TEXT NOT NULL CHECK (source IN ('france_travail', 'adzuna')),
  id_source   TEXT,
  hash_offre  TEXT NOT NULL UNIQUE,
  titre       TEXT NOT NULL,
  entreprise  TEXT,
  lieu        TEXT,
  code_dept   TEXT REFERENCES departements(code_dept),
  code_rome   TEXT REFERENCES metiers_rome(code_rome),
  categorie   TEXT,
  contrat     TEXT CHECK (contrat IN
('CDI', 'CDD', 'Intérim', 'Libéral', 'Alternance', 'Autre', 'Non renseigné')),
  temps_travail TEXT,
```

```

experience      TEXT,
salaire_min     REAL CHECK (salaire_min IS NULL OR salaire_min BETWEEN 10000 AND 300000),
salaire_max     REAL CHECK (salaire_max IS NULL OR salaire_max BETWEEN 10000 AND 300000),
salaire_moyen   REAL,
salaire_impute  INTEGER NOT NULL DEFAULT 0 CHECK (salaire_impute IN (0, 1)),
salaire_source  TEXT,
rome_source     TEXT CHECK (rome_source IS NULL OR rome_source IN ('france_travail',
'intitule', 'categorie')),
date_publication TEXT,
description     TEXT,
url            TEXT,
date_import     TEXT NOT NULL
);

```

Contrainte / index	Rôle
hash_offre UNIQUE	Garantit l'absence de doublon, même si le pipeline est relancé ; le schéma est recréé à chaque exécution : même base à chaque run (INSERT OR IGNORE)
FOREIGN KEY code_dept, code_rome	Intégrité référentielle, activée par PRAGMA foreign_keys = ON
CHECK source, contrat, salaires, format ROME	Rejette toute valeur hors du domaine autorisé dès l'insertion
6 index	code_dept, code_rome, source, contrat, date_publication, statut : les filtres de l'API

Volumétrie

Table	Lignes
departements	8
metiers_rome	1 514
offres	62 435
indicateurs_tension	1 514

Conformité RGPD

Le registre complet est versionné dans [docs/rgpd.md](#). Points clés :

Rubrique	Mesure
Finalité	Statistiques agrégées sur le marché du travail francilien
Base légale	Intérêt légitime — données publiées par les employeurs, collectées via les API officielles
Personnes concernées	Aucun candidat. Seules les coordonnées de recruteurs peuvent figurer dans le texte libre
Minimisation	Aucune donnée de candidat ; description tronquée à 300 caractères (France Travail) ou 500 (Adzuna)
Masquage	E-mails et téléphones remplacés par [email] / [tel] avant stockage
Conservation	12 mois glissants — purger_offres_anciennes() à chaque exécution (807 offres purgées : publiées plus de 365 jours avant l'offre la plus récente)

Rubrique	Mesure
Sécurité	Secrets dans <code>.env</code> ; base ouverte en lecture seule par l'API ; requêtes paramétrées ; clé API
Contrôle	Test automatisé <code>test_aucune_donnee_personnelle</code>

7. C5 — Mettre à disposition les données via une API

L'API REST est développée avec **FastAPI**. Elle expose les offres et les indicateurs de tension en lecture seule, avec une documentation interactive générée automatiquement (OpenAPI / Swagger).

Endpoints

Verbe	Endpoint	Accès	Description
GET	/health	Public	Disponibilité, nombre d'offres et de métiers, date de mise à jour
GET	/offres	Clé	Liste paginée ; filtres departement, code_rome, source, contrat, q (mot-clé)
GET	/offres/{offre_id}	Clé	Détail d'une offre (404 si inexistante)
GET	/metiers	Clé	Indicateurs de tension triés, filtre statut
GET	/metiers/{code_rome}	Clé	Fiche métier : répartition par département et par contrat
GET	/stats/departements	Clé	Volume, densité (offres / 100 km ²), part de CDI, salaire médian, coordonnées
GET	/stats/tension	Clé	Nombre de métiers et d'offres par statut

Sécurité

Risque	Mesure
Accès non autorisé	En-tête X-API-Key obligatoire (sauf /health) → HTTP 403 ; clé lue dans la variable d'environnement PPMT_API_KEY (l'API refuse de démarrer si elle est absente)
Injection SQL	Requêtes paramétrées (?) ; aucun paramètre concaténé dans le SQL
Paramètres invalides	Validation Pydantic : motif du département, du code ROME, bornes de pagination → HTTP 422
Extraction massive	limit ≤ 500 (50 par défaut), pagination par offset ; clé API obligatoire
Modification des données	Verbe GET uniquement (405 sinon) ; connexion SQLite en mode lecture seule (mode=ro)
Appels cross-origin	CORS limité au verbe GET

api/main.py – contrôle de la clé

```
_api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)

def verifier_cle(key: Optional[str] = Security(_api_key_header)) -> None:
    if key != API_KEY:
        raise HTTPException(status_code=403, detail="Clé API invalide ou manquante")
```

api/main.py – endpoint /offres (filtres + pagination, SQL paramétré)

```
@app.get("/offres", response_model=PageOffres, tags=["Offres"],
dependencies=[Depends(verifier_cle)],
summary="Liste paginée et filtrable des offres")
def lister_offres(
    departement: Optional[str] = Query(None, pattern=r"^(75|77|78|91|92|93|94|95)$",
description="Code département"),
```

```

code_rome: Optional[str] = Query(None, pattern=r"^[A-N]\d{4}$", description="Code ROME, ex.
J1502"),
source: Optional[Source] = None,
contrat: Optional[str] = Query(None, description="CDI, CDD, Intérim, Libéral, Autre..."),
q: Optional[str] = Query(None, min_length=2, description="Mot-clé dans l'intitulé"),
limit: int = Query(50, ge=1, le=500),
offset: int = Query(0, ge=0),
):
    filters, params = [], []
    for col, val in (("code_dept", departement), ("code_rome", code_rome),
                    ("source", source.value if source else None), ("contrat", contrat)):
        if val:
            filters.append(f"{col} = ?")
            params.append(val)
    if q:
        filters.append("titre LIKE ?")
        params.append(f"%{q}%")
    where = f"WHERE {' AND ' .join(filters)}" if filters else ""
    with db() as conn:
        total = conn.execute(f"SELECT COUNT(*) FROM offres {where}", params).fetchone()[0]
        res = lignes(conn, f""SELECT id, source, titre, entreprise, lieu, code_dept, code_rome,
contrat,
                        salaire_moyen, salaire_impute, date_publication, url
FROM offres {where}
ORDER BY date_publication DESC LIMIT ? OFFSET ?""", params +
[limit, offset])
    return {"total": total, "limit": limit, "offset": offset, "resultats": res}

```

Documentation Swagger

127.0.0.1:8000/docs

API PPMT — Métiers en tension Île-de-France 2.0.0 OAS 3.1

OpenAPI JSON

Offres d'emploi France Travail + Adzuna (8 départements franciliens) et indicateurs de tension par code ROME. Authentification : en-tête X-API-Key.

Authorize

Service	Method	Endpoint	Description	Protected
Service	GET	/health	Disponibilité du service (public)	Public
Offres	GET	/offres	Liste paginée et filtrable des offres	Protected
	GET	/offres/{offre_id}	Détail d'une offre	Protected
Tension	GET	/metiers	Indicateurs de tension par métier (tri décroissant)	Protected
	GET	/metiers/{code_rome}	Fiche métier : tension, départements, contrats	Protected
Statistiques	GET	/stats/departements	Volume d'offres, densité, part de CDI, salaire médian et coordonnées par département	Protected
	GET	/stats/tension	Répartition des métiers par statut de tension	Protected

Figure 6 — Documentation interactive générée par FastAPI (/docs)

Légende : le cadenas signale un endpoint protégé par la clé API ; /health reste public.

Preuve d'exposition des données

Appels réels sur l'API lancée localement (`uvicorn api.main:app`) avec la base produite par le pipeline :

Contrôles d'accès et de validation – captures de l'API lancée localement (/health public ; /metiers sans clé : refus 403), puis appel avec clé

localhost:8000/health HTTP 200 · public

```
{ "status": "ok", "nb_offres": 62435, "nb_metiers": 1514, "derniere_mise_a_jour": "2026-10-06T15:58:12" }
```

localhost:8000/metiers HTTP 403 · sans clé

```
"detail": "Clé API invalide ou manquante" }
```

GET /offres Liste paginée et filtrable des offres

Parameters

Name Description

departement Code département

string | (string | null) 13

(query) pattern: "^(75|77|78|91|92|93|94|95)\$"

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  http://localhost:8000/offres?departement=13 --limit=50 --offset=0 \
  -H 'accept: */*' \
  -H 'x-api-key: ppmt-demo-2025'
```

Request URL

```
http://localhost:8000/offres?departement=13 --limit=50 --offset=0
```

Server response

Code Details

422 Error: Unprocessable Entity

Response body

```
{
  "detail": [
    {
      "type": "string_pattern_mismatch",
      "loc": [
        "query",
        "departement"
      ],
      "msg": "String should match pattern \"^(75|77|78|91|92|93|94|95)$\"",
      "input": "13",
      "ctx": {
        "pattern": "^(75|77|78|91|92|93|94|95)$"
      }
    }
  ]
}
```

Démarrage sans clé : l'API refuse de se lancer si PPMT_API_KEY n'est pas définie (erreur explicite)

```
File "/home/djell/PPMT/api/main.py", line 26, in <module>
    raise RuntimeError("Variable d'environnement PPMT_API_KEY absente : définissez-la avant de lancer l'API ")
RuntimeError: Variable d'environnement PPMT_API_KEY absente : définissez-la avant de lancer l'API (ex. : export PPMT_API_KEY=<votre clé>).
% ➔ PPMT git:(main)
```

Fiche métier « Aide ménager / Aide ménagère à domicile » (répartitions tronquées à 3 lignes)

```
$ curl -H "X-API-Key: $PPMT_API_KEY" http://localhost:8000/metiers/K1304
{
  "code_rome": "K1304",
  "libelle": "Aide ménager / Aide ménagère à domicile",
  "nb_offres_ft": 1426,
  "nb_offres_adzuna": 105,
  "nb_offres_total": 1531,
  "nb_entreprises": 321,
  "part_cdi": 81.1,
  "salaire_median": 22477.0,
  "part_salaire_affiche": 48.1,
  "indice_tension": 3842.4,
  "statut": "TRES EN TENSION",
  "repartition_departements": [
    { "code_dept": "78", "nom": "Yvelines", "nb_offres": 326 },
    { "code_dept": "91", "nom": "Essonne", "nb_offres": 233 },
    { "code_dept": "92", "nom": "Hauts-de-Seine", "nb_offres": 229 }
  ],
  "repartition_contrats": [
    { "contrat": "CDI", "nb_offres": 1241 },
    { "contrat": "CDD", "nb_offres": 191 },
    { "contrat": "Non renseigné", "nb_offres": 80 }
  ]
}
```

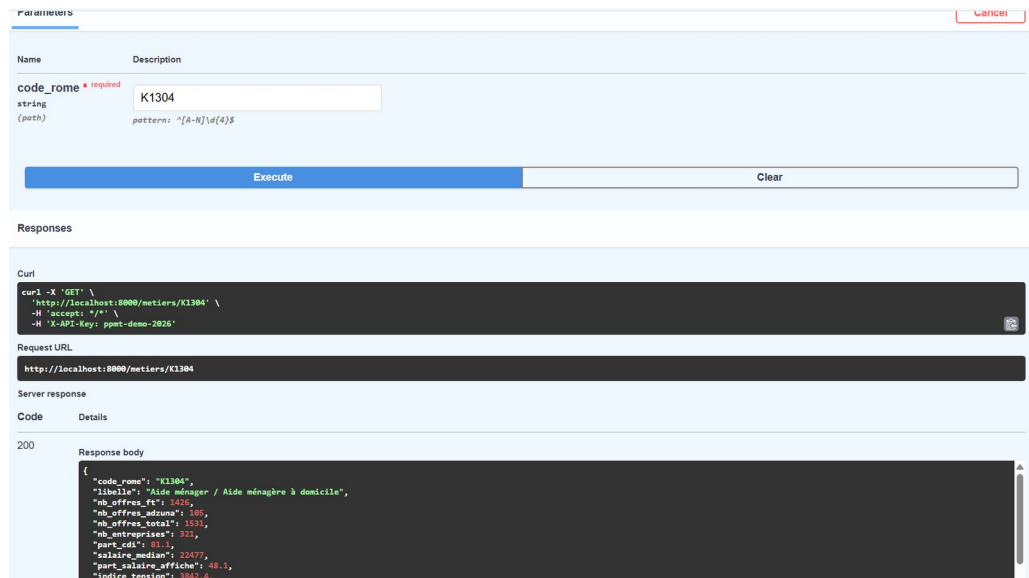


Figure 7 — M\u00eame appel depuis Swagger (« Try it out »)

D\u00e9ploiement

Dockerfile – brouillon de d\u00e9ploiement (non valid\u00e9)

```
# Image de l'API PPMT (C5) – d\u00e9ployable sur Google Cloud Run, Render, Railway...
FROM python:3.11-slim
WORKDIR /app
COPY api/requirements-api.txt .
RUN pip install --no-cache-dir -r requirements-api.txt
COPY api/ ./api/
COPY data/ppmt.db ./data/ppmt.db
ENV PORT=8080 PPMT_DB=/app/data/ppmt.db
# PPMT_API_KEY doit \u00eatre fournie \u00e0 l'ex\u00e9cution (secret), ex. : docker run -e PPMT_API_KEY=...
CMD uvicorn api.main:app --host 0.0.0.0 --port ${PORT}
```

Ce Dockerfile figure dans le d\u00e9p\u00f4t mais n'est pas op\u00e9rationnel \u00e0 ce stade : il n'est pas utilis\u00e9 pour la d\u00e9monstration (l'API se lance avec uvicorn, §7) et relève de la perspective de d\u00e9ploiement (§10). Dans ce cas, la cl\u00e9 API serait inject\u00e9e comme secret au lancement.

8. Tests et qualité

`tests/test_pipeline.py` contient **57 tests** exécutés avec `pytest tests/ -v`. Les appels aux API externes sont simulés (objets `FakeSession`) pour tester la pagination et la reprise sur erreur sans dépendre du réseau ni consommer de quota.

Catégorie	Tests	Exemples de vérifications
C1 — collecte (API simulées)	4	arrêt à la dernière page ; plafond 3 150 offres ; reprise après HTTP 429 ; aplatissement du JSON. La cascade par dates n'a pas encore de test unitaire : elle est contrôlée par comparaison avec le compteur Content-Range
C2/C3 — règles de préparation	31	conversion des salaires ; cascade de médianes ; codes ROME par intitulé ; référentiel des communes ; libellés de salaire ; départements ; contrats ; masquage des e-mails et téléphones ; priorité France Travail
C4 — base de données	13	aucun salaire vide ; départements géolocalisés ; tables présentes ; intégrité référentielle ; zéro doublon ; périmètre IDF ; aucun e-mail ni téléphone en clair ; CHECK salaires ; ITM moyen = 100
C5 — API	9	/health public ; 403 sans clé ; filtres ; 422 si paramètre invalide ; pagination ; 404 ; 405 sur POST

Capture d'écran – exécution de `pytest tests/ -v` : 57 tests réussis en 10,26 s

```

→ PPMT git:(main) pytest tests/ -v
tests/test_pipeline.py::test_stats.departements_avec_coordonnees PASSED
tests/test_pipeline.py::test_offres_filtre_departement PASSED
tests/test_pipeline.py::test_parametre_invalide_422 PASSED
tests/test_pipeline.py::test_pagination PASSED
tests/test_pipeline.py::test_metiers_tries_par_tension PASSED
tests/test_pipeline.py::test_fiche_metier_et_404 PASSED
tests/test_pipeline.py::test_api_lecture_seule PASSED

===== 57 passed in 10.26s =====

% → PPMT git:(main)

```

[Artefact]

[89%]
[91%]
[92%]
[94%]
[96%]
[98%]
[100%]

9. Service numérique et ouverture Machine Learning

Dashboard Streamlit

Le dashboard (`streamlit run webapp/app.py`) s'adresse aux utilisateurs non techniques. Il se présente de deux façons : en local sur la base figée du 6 octobre (choix 1, démonstration principale) ou en ligne sur Streamlit Cloud (choix 2, secours : <https://ppmt-claire.streamlit.app>). Il propose six onglets avec filtres par département, contrat et secteur :

Onglet	Contenu
Vue d'ensemble	Indicateurs clés, répartition des statuts, top 10 des métiers en tension
Secteurs	Volumes par secteur, contrats France Travail et Adzuna
Compétences & tech	Compétences techniques et comportementales extraites des offres des métiers en tension
Formations & recrutement	Niveaux requis, part de CDI / CDD
Métiers en tension	Statuts, carte des départements dessinée à partir du fichier GeoJSON
Salaires	Comparaison France Travail / Adzuna par métier, secteur et département

Ouverture Machine Learning : classer les métiers en tension

Approche : classification « en tension » (indice > 100) à partir du **profil** des offres uniquement — part de CDI, CDD et intérim, salaire, part de salaires non affichés, expérience demandée, longueur des descriptions, famille ROME — sans aucune variable de volume. Les 717 métiers ayant au moins 10 offres (dont 46 % en tension) sont évalués en validation croisée stratifiée à 5 plis. Trois modèles sont comparés ; le modèle retenu est celui qui obtient la meilleure AUC. Pour repère, un classement au hasard obtient 0,51.

Modèle	ROC-AUC	F1	Exactitude équilibrée
Régression logistique	0,61	0,56	0,56
Random Forest	0,87	0,76	0,78
XGBoost	0,89	0,79	0,80

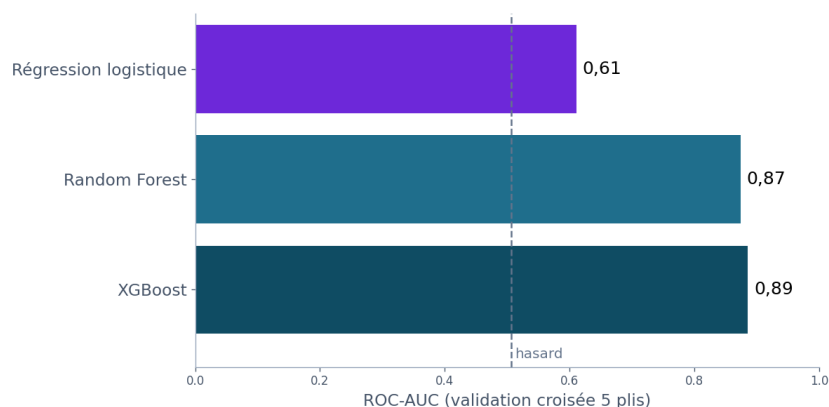


Figure 8 — Comparaison des modèles (pointillés : classement au hasard)

XGBoost est retenu (AUC 0,89, F1 0,79) : le profil des offres contient un signal réel. Les variables les plus explicatives sont la longueur des descriptions (importance 0,17), la part d'intérim (0,09) et de CDD (0,07), la part d'offres ouvertes aux débutants (0,06) et la famille de métiers (familles C, K et J : banque-assurance-immobilier, services à la personne, santé) : l'importance d'une variable n'indique pas le sens de son effet, et la longueur des

descriptions reflète surtout la source (300 caractères conservés pour France Travail, 500 pour Adzuna) : c'est un signal exploratoire, pas une explication. Sans cette variable, l'AUC de XGBoost passe de 0,886 à 0,795 : le signal reste réel (hasard : 0,51) mais plus modeste. Cette partie ne relève pas du bloc BC01 ; elle ouvre sur le bloc suivant.

10. Bilan et perspectives

Correspondance avec le référentiel

Compétence	Livrables	Preuves
C1 — Collecte automatisée	<code>src/collect.py</code>	2 API (OAuth2, clé) + 2 fichiers de référence (contours, communes) réutilisés en base et dans l'API ; cascade de découpage (99,5 % des offres France Travail) ; reprise ; journaux ; 4 tests
C2 — Préparation	<code>src/prepare.py</code>	10 problèmes traités (plusieurs méthodes comparées), cascade de médianes pour les salaires ; schéma commun ; RGPD ; tests unitaires
C3 — Agrégation	<code>SQL_INDICATEURS</code> (store.py)	8 indicateurs par métier : SQL (GROUP BY, fonction fenêtre) + salaire médian
C4 — Base de données	<code>src/store.py</code> , <code>data/ppmt.db</code> , <code>docs/rgpd.md</code>	MPD 4 tables, FK, CHECK, UNIQUE, index ; registre RGPD ; purge
C5 — API	<code>api/main.py</code>	7 endpoints, clé API, Swagger, validation, tests 403/404/405/422

Chiffres clés

Indicateur	Valeur
Offres en base	62 435 (France Travail 58 301 · Adzuna 4 134)
Métiers ROME couverts	1 514
Métiers « très en tension »	235
Départements	8
Tests automatisés	57 (100 % réussis)
Durée d'exécution du pipeline (hors collecte)	≈ 35 secondes (collecte France Travail : ≈ 6 min 40 s)

Limites

- L'indice de tension ne mesure que la demande des employeurs ; il manque le nombre de demandeurs d'emploi par métier.
- La correspondance catégorie Adzuna → ROME est grossière (25 catégories, un code chacune) : elle surreprésente certains métiers ; 2 110 offres Adzuna (51 %) restent sans code et sont exclues de l'indice.
- Une seule collecte complète (octobre 2026) : pas encore de série temporelle. La base est reconstruite à chaque exécution ; l'historique n'existe que dans les fichiers bruts datés (data/raw).
- Adzuna : seulement 12 500 offres lues sur 220 238 annoncées (plafond de 250 pages fixé dans notre collecte) ; Adzuna sert d'échantillon complémentaire, France Travail porte le volume.
- La cascade de collecte France Travail n'est pas encore couverte par un test unitaire, et la collecte n'est pas planifiée automatiquement (lancement manuel).

Perspectives

- **Offre / demande** : intégrer les demandeurs d'emploi par code ROME (statistiques France Travail / DARES) pour calculer un vrai ratio de tension.
- **Historisation** : conserver un instantané quotidien des indicateurs pour prévoir la tension à 3 mois (modèles de séries temporelles).
- **Déploiement** : rendre le Dockerfile opérationnel, puis publier l'API sur Google Cloud Run (le dashboard dispose déjà d'une version en ligne de secours sur Streamlit Cloud, <https://ppmt-claire.streamlit.app>) ; planification quotidienne (cron, puis Airflow si le nombre de tâches le justifie).
- **Collecte Adzuna** : relever ou supprimer le plafond de 250 pages (ADZUNA_MAX_PAGES) pour lire une plus grande part des 220 238 offres annoncées, après vérification du quota de l'API et du surcroît de doublons avec France Travail.
- **Passage à l'échelle** : extension aux autres régions.

Annexes

A — Lancement rapide

```
git clone https://github.com/Clairelucie/PPMT.git && cd PPMT
pip install -r requirements.txt
cp .env.example .env                # renseigner les identifiants France Travail et Adzuna
python run_pipeline.py --collect     # C1 → C4 (sans --collect : repart des fichiers bruts déjà
collectés dans data/raw)
pytest tests/ -v                    # 57 tests
uvicorn api.main:app --reload        # API → http://localhost:8000/docs
streamlit run webapp/app.py          # dashboard → http://localhost:8501
```

B — Dépendances

```
# PPMT – dépendances (RNCP37827BC01 – Artefact 2026)
# Collecte & préparation (C1 → C3)
pandas>=2.0
numpy>=1.23
requests>=2.28
python-dotenv>=0.21
# API (C5)
fastapi>=0.110
uvicorn[standard]>=0.29
pydantic>=2.5
# Tests
pytest>=8.0
httpx>=0.27
# Machine Learning (expérimentation)
scikit-learn>=1.3
xgboost>=1.7
joblib>=1.2
# Dashboard
streamlit>=1.30
plotly>=5.10
```

C — Clé API de démonstration

Clé utilisée pour la démonstration devant le jury : **ppmt-demo-2026**. Cette clé est publique dans ce dossier et sera révoquée après la soutenance. Elle est lue dans la variable d'environnement **PPMT_API_KEY** ; elle n'a aucune valeur par défaut dans le code : si la variable est absente, l'API refuse de démarrer. En production, une clé secrète est fournie à l'exécution.

D — Glossaire

Terme	Définition
ROME	Répertoire Opérationnel des Métiers et des Emplois (France Travail) — un code par métier, ex. J1502
ITM	Indice de tension PPMT : volume d'offres d'un métier rapporté au métier moyen (× 100)
BMO	Enquête Besoins en Main-d'Œuvre de France Travail
DARES	Direction de l'animation de la recherche, des études et des statistiques (ministère du Travail)
OAuth2	Protocole d'autorisation ; ici, flux « client credentials » entre applications

Terme	Définition
RGPD	Règlement général sur la protection des données

E — Cascade de collecte France Travail



Figure E1 — Cascade de collecte France Travail (plafond de 3 150 offres par requête)

Dossier rédigé en octobre 2026 — Compétences RNCP37827BC01 (C1 à C5)